

FIAP — Faculdade de Informática e Administração Paulista
Graduação ON em Inteligência Artificial
Atividade Substitutiva Global Solution 2 — 2026.1 | Fase 7

OrbitalGuard

Classificação de Detritos Espaciais com CNN + Monitoramento Orbital

Aluno	Alan Robin Santos
RM	567437
Data	16 de junho de 2026
GitHub	https://github.com/Yole87/Atividades-FIAP/tree/FIAP_IA_Online/Fase%207/Atividade%20Substitutiva%20Global%20Solution
Vídeo	https://youtu.be/jC8Nrqa1koA

1. Introdução

1.1 Contextualização — A Nova Economia Espacial

A exploração espacial representa hoje uma das maiores oportunidades tecnológicas e econômicas do século XXI. Satélites de observação monitoram continuamente a superfície do planeta, enquanto o crescimento acelerado de lançamentos cria um novo desafio crítico: a proliferação de detritos orbitais. Estima-se que mais de 27.000 objetos rastreados estejam em órbita, representando risco real para satélites ativos e missões tripuladas.

1.2 O Problema

A identificação e classificação de detritos espaciais é um problema de visão computacional e aprendizado de máquina ainda em aberto. Soluções existentes dependem de radares terrestres caros e dados proprietários. O OrbitalGuard propõe democratizar essa inteligência usando CNN própria, dados públicos da ISS e hardware acessível como o ESP32.

1.3 A Pergunta da Global Solution

Como a Inteligência Artificial e as tecnologias digitais podem transformar a nova economia espacial e gerar impacto positivo na Terra?

O OrbitalGuard responde conectando uma CNN treinada para classificar detritos orbitais com monitoramento em tempo real via ESP32, integração com a API da ISS e arquitetura serverless AWS.

2. Desenvolvimento

2.1 Arquitetura da Solução

A solução é composta por cinco módulos integrados, todos compartilhando a definição da CNN através do módulo `model.py`:

Modulo	Tecnologia	Função
Pipeline de ML	PyTorch + scikit-learn	Treina CNN para classificar 4 tipos de objetos orbitais em imagens 64x64
Dashboard	Streamlit + Plotly	Interface interativa com simulador de risco orbital e CNN ao vivo
Sensor IoT	ESP32 + MPU6050 + MQTT	Detecta vibração e impacto de micro detritos em tempo real
Visão Computacional	OpenCV + PIL + PyTorch	Classifica objetos em cena orbital via grid de imagens sintéticas
AWS Lambda	Python + Open Notify API + CNN	API REST serverless que classifica o detrito mais próximo via CNN

Fluxo de dados:

```
Imagens Sintéticas > CNN (PyTorch) > Classificação de Detritos > Dashboard
Open Notify API   > Lambda > CNN > Risco de Colisão Orbital > Dashboard
ESP32 + MPU6050  > MQTT (HiveMQ) > Enriquecimento local
```

2.2 Modulo 1 — Pipeline de Machine Learning (CNN PyTorch)

2.2.1 Geracao do Dataset Sintetico

O dataset foi gerado proceduralmente com 800 imagens (200 por classe) usando PIL e NumPy. A estratégia central foi usar paletas de cor com variação de mais ou menos 60 pontos RGB entre classes, forçando a CNN a aprender características de forma e estrutura geométrica em vez de cor:

- `satellite_ativo`: corpo retangular simétrico + DOIS painéis solares opostos + antena
- `detrito metálico`: polígono irregular convexo com bordas afiadas e reflexo metálico pontual
- `fragmento rochoso`: contorno orgânico com alta variação de raios + textura interna densa
- `satellite_inativo`: corpo assimétrico + apenas UM painel + fragmento solto separado

2.2.2 Treinamento com Data Augmentation

O treinamento aplica data augmentation em cada batch, forçando o modelo a aprender características robustas de forma:

```
RandomHorizontalFlip(p=0.5)      # objeto em qualquer lado
RandomVerticalFlip(p=0.5)
RandomRotation(degrees=45)       # qualquer orientação
ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3, hue=0.15)
RandomAffine(degrees=0, translate=(0.1,0.1), scale=(0.85,1.15))
```

2.2.3 Arquitetura da CNN (`model.py` compartilhado)

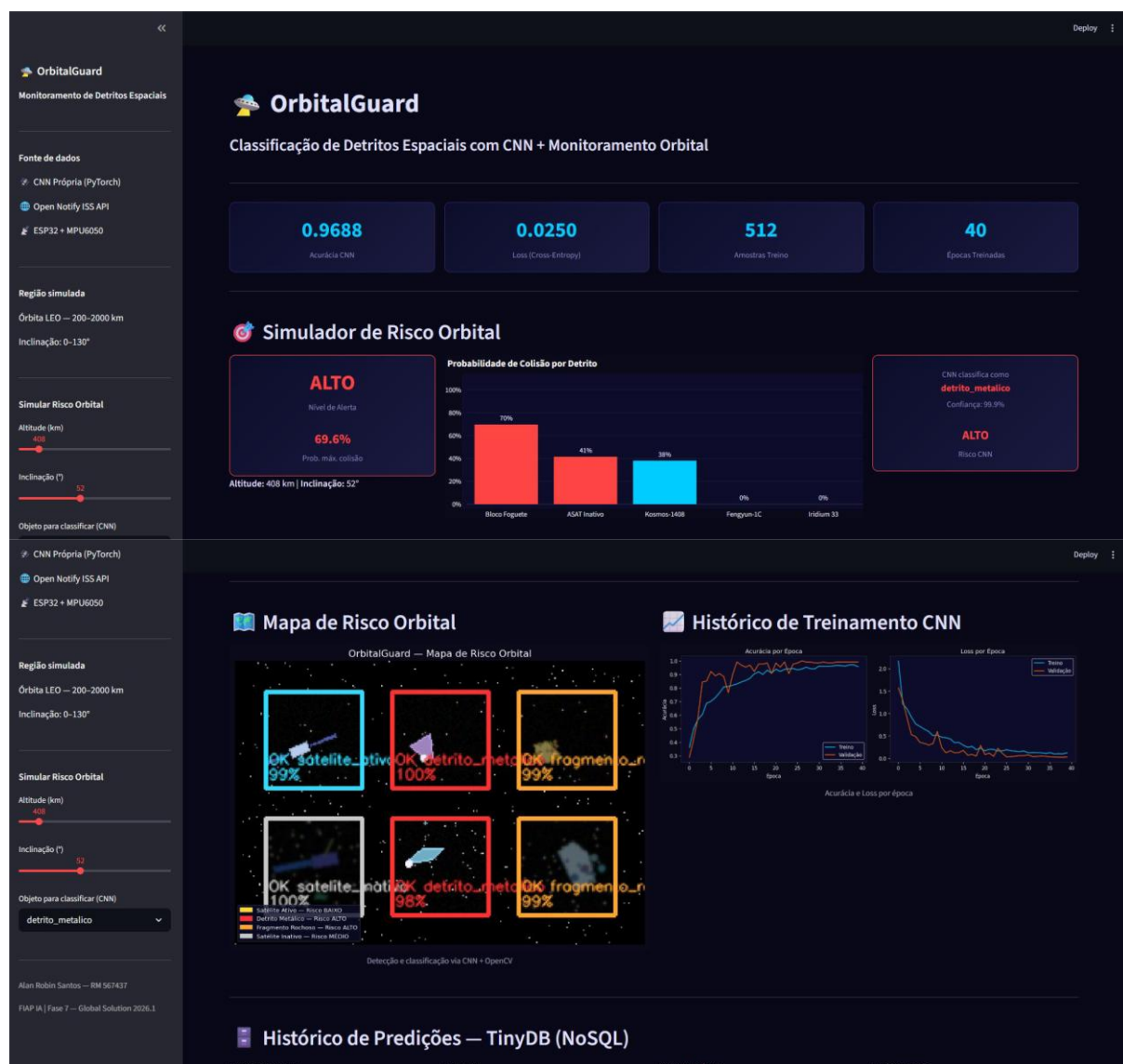
A OrbitalCNN usa 3 blocos Conv+BatchNorm+ReLU+MaxPool seguidos de classificador Dense+Dropout. O modulo model.py e compartilhado entre treinar_cnn.py, analisar_orbita.py, lambda_function.py e app.py:

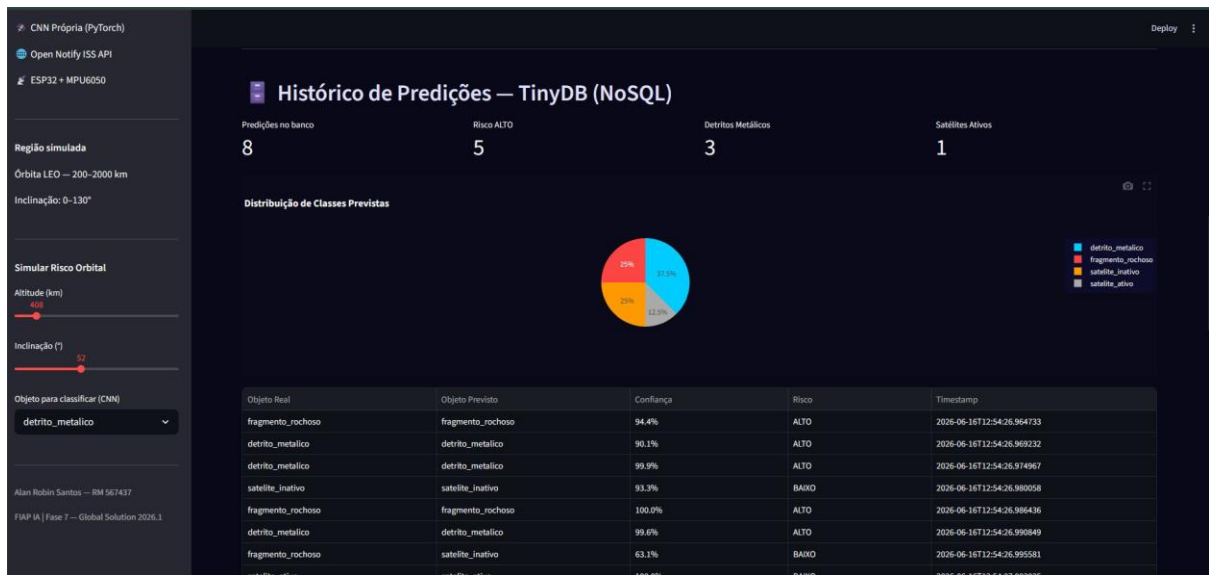
```
Conv2d(3>32) + BN + ReLU + MaxPool(2)
Conv2d(32>64) + BN + ReLU + MaxPool(2)
Conv2d(64>128) + BN + ReLU + MaxPool(2)
Flatten > Linear(8192>256) > ReLU > Dropout(0.4) > Linear(256>4)
```

Otimizador: Adam (lr=1e-3, weight_decay=1e-4) | Scheduler: CosineAnnealingLR | Checkpoint: melhor val_acc salvo a cada epoca.

2.3 Modulo 2 — Dashboard Interativo (Streamlit + Plotly)

O dashboard oferece métricas do modelo em tempo real, simulador orbital interativo com CNN ao vivo, mapa de risco com bounding boxes por classe e historico TinyDB. O seletor de objeto na sidebar classifica instantaneamente com o modelo carregado.





2.4 Modulo 3 — Sensor IoT de Campo (ESP32 + MPU6050)

O nó de sensor usa ESP32 DevKit V1 com acelerometro/giroscopio MPU6050 (I2C) para detectar padrões de vibração simulando impacto de micro detritos. O firmware C++ coleta leituras a cada 2 segundos e publica via MQTT no broker HiveMQ.

Componente	Conexão	Função
MPU6050	GPIO 21 (SDA) / GPIO 22 (SCL)	Acelerometro 3 eixos + giroscopio
LED Alerta	GPIO 2 (embutido)	Feedback visual de nível de impacto
WiFi ESP32	Integrado	Publicação MQTT via HiveMQ

Payload MQTT — tópico orbitalguard/sensor/vibration01:

```
{"sensor_id":"OrbitalGuard-ESP32-001","accel_magnitude":1.23,"classificacao":"MODERADO","nivel_risco":"MEDIO","impactos_sessao":2}
```

2.5 Modulo 4 — Visao Computacional

O modulo monta uma cena orbital como grid 3x2 de objetos 64x64 gerados pelos mesmos geradores do dataset, garantindo consistência visual com o treino. Cada célula é classificada individualmente pela CNN sem redimensionamento, preservando as features de forma aprendidas.

Nota técnica: a abordagem usa grid sintético como POC. A classificação opera diretamente nas células de 64x64 nativas — o mesmo tamanho do treino — evitando degradação por redimensionamento.

2.6 Modulo 5 — AWS Lambda (Arquitetura Serverless)

A função Lambda recebe coordenadas orbitais, busca a posição real da ISS via Open Notify API, identifica o detrito mais proximo por distancia orbital e classifica o TIPO desse detrito usando a CNN treinada — conectando os módulos de forma real.

Fluxo: Cliente > API Gateway > Lambda > CNN (PyTorch) + Open Notify API > S3 + CloudWatch

Endpoint: GET https://api.orbitalguard.io/risco?altitude_km=408&inclinacao=51.6

3. Resultados Obtidos

3.1 Desempenho do Modelo CNN

Métrica	Valor	Interpretação
Acurácia (teste)	96.88%	O modelo classifica corretamente 97 em 100 objetos
Loss (Cross-Entropy)	0.0250	Convergência estável após 40 épocas
Amostras de treino	512	80% de 640 (split 80/20 + 20% validação)
Amostras de teste	160	Dados nunca vistos pelo modelo
Épocas treinadas	40	Com CosineAnnealingLR e early checkpoint
Melhor val_acc	100% (epoca 27)	Checkpoint salvo — teste avaliado com este modelo

3.2 Relatório por Classe

Classe	Precision	Recall	F1-Score	Suporte
satelite_ativo	0.98	1.00	0.99	40
detrito metálico	1.00	0.95	0.97	40
fragmento rochoso	1.00	0.93	0.96	40
satelite_inativo	0.91	1.00	0.95	40
accuracy			0.97	160
macro avg	0.97	0.97	0.97	160

O erro tecnico mais relevante foi fragmento rochoso confundido com satelite_inativo (recall 0.93), o que é esperado: ambas as classes tem formas irregulares escuras. Isso evidencia que o modelo aprendeu características de forma, não de cor.

3.3 Saídas dos Terminais

Terminal — python ml/gerar_dataset.py: 800 imagens, 4 classes, seed aleatória

```
[Dataset] Seed: 69870
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python ml/generar_dataset.py
>>
[Dataset] Seed: 69870
[Dataset] Seed: 69870
[Dataset] Estratégia variação de cor ampla + CNN aprende FORMA, não cor
[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard>

[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json
[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[Dataset] 800 Imagens geradas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\daset/
satelite ativo: 200 amostras
detrito metalico: 200 amostras
fragmento rochoso: 200 amostras
satelite inativo: 200 amostras
[LABELS] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\labels.json
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard>
```

Terminal — python ml/treinar_cnn.py: 40 épocas com data augmentation

```
# Problems Output Debug Console Terminal Ports
```

```
# PS D:\Atividades-FIAP\FIAP_IA_Online\Fase 7\Atividade Substitutiva Global_Solution\OrbitalGuard> python ml/trainer_cnn.py
```

```
[DEVIC] Usando: cpu
```

```
[AUGMENTATION] Flip H/V + Rotação 45° + ColorJitter + Affine - CNN aprende FORMA
```

```
[DADOS] Carregando imagens do dataset...
```

```
[DADOS] 888 imagens carregadas
```

```
[SPLIT] Treino: 512 | Val: 128 | Teste: 160
```



```
[CNN] Iniciando treinamento - 40 épocas com data augmentation...
```

época	loss:	Acc:	loss:	Val Acc:
01/40	2.1702	0.3613	1.5701	0.2891
02/40	1.2188	0.5480	1.2038	0.4210
03/40	1.1035	0.5684	0.9074	0.5781
04/40	0.9206	0.6035	0.5287	0.8438
05/40	0.7655	0.6875	0.4833	0.8516
06/40	0.7107	0.7012	0.3533	0.9219
07/40	0.6500	0.7205	0.3330	0.8906
08/40	0.5963	0.7656	0.2936	0.9062
09/40	0.5035	0.8086	0.3276	0.8828
10/40	0.5195	0.8105	0.5973	0.7656
11/40	0.4719	0.8203	0.2383	0.8984
12/40	0.4604	0.8301	0.1225	0.9922
13/40	0.4234	0.8436	0.1764	0.9688
14/40	0.3467	0.8555	0.1216	0.9531
15/40	0.3523	0.8711	0.1294	0.9688
16/40	0.2917	0.9043	0.1776	0.9219
17/40	0.2458	0.9199	0.0896	0.9766
18/40	0.2652	0.9084	0.0884	0.9766
19/40	0.1942	0.9316	0.0511	0.9844
20/40	0.2286	0.9182	0.2863	0.9062
21/40	0.1687	0.9395	0.0826	0.9844
22/40	0.2028	0.9219	0.1179	0.9531
23/40	0.1985	0.9375	0.0433	0.9922
24/40	0.1628	0.9414	0.0237	0.9862
25/40	0.1677	0.9434	0.1146	0.9766
26/40	0.1905	0.9336	0.0335	0.9844
27/40	0.1702	0.9395	0.0383	1.0000
28/40	0.1587	0.9531	0.0446	0.9922
29/40	0.1405	0.9414	0.0620	0.9922
30/40	0.1633	0.9414	0.0607	0.9844
31/40	0.1231	0.9609	0.0786	0.9844
32/40	0.1289	0.9590	0.0385	0.9922
33/40	0.1271	0.9590	0.0414	0.9844
34/40	0.1234	0.9609	0.0513	0.9844
35/40	0.1077	0.9668	0.0434	0.9922
36/40	0.1261	0.9648	0.0348	0.9922
37/40	0.1006	0.9609	0.0288	0.9922
38/40	0.1041	0.9707	0.0268	0.9922
39/40	0.1014	0.9707	0.0250	0.9922
40/40	0.1224	0.9576	0.0388	0.9922


```
[MODELO] Carregando melhor checkpoint (Val Acc: 1.0000)...
```



```
[MODELO] Acurácia no teste: 0.9688
```

```
[NOTA] Acurácia < 100% esperada - CNN aprende forma com cores sobrepostas + augmentation
```



```
[RELATÓRIO] Por classe:
```


Terminal — Relatório por classe e exemplos de previsão do conjunto de teste

```
Problems Output Debug Console Terminal Ports

[RELATÓRIO] Por classe:
precision recall f1-score support
satelite_ativo 0.98 1.00 0.99 40
detrito_metalico 1.00 0.95 0.97 40
fragmento_rochoso 1.00 0.93 0.96 40
satelite_inativo 0.91 1.00 0.95 40

accuracy 0.97 160
macro avg 0.97 0.97 0.97 160
weighted avg 0.97 0.97 0.97 160

[MÉTRICAS] Salvas em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\metricas.json
[TINYDB] Banco limpo e métricas inseridas

[PREVISÃO] Exemplos do conjunto de teste:
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 94.36% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 90.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.89% | Risco: ALTO
✓ Real: satelite_inativo Previsto: satelite_inativo Confiança: 93.38% | Risco: BAIXO
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 100.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.62% | Risco: ALTO
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 94.36% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 90.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.89% | Risco: ALTO
✓ Real: satelite_inativo Previsto: satelite_inativo Confiança: 93.38% | Risco: BAIXO
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 100.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.62% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.89% | Risco: ALTO
✓ Real: satelite_inativo Previsto: satelite_inativo Confiança: 93.38% | Risco: BAIXO
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 100.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.62% | Risco: ALTO
✓ Real: fragmento_rochoso Previsto: fragmento_rochoso Confiança: 100.00% | Risco: ALTO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.62% | Risco: ALTO
X Real: fragmento_rochoso Previsto: satelite_inativo Confiança: 63.10% | Risco: BAIXO
✓ Real: detrito_metalico Previsto: detrito_metalico Confiança: 99.62% | Risco: ALTO
X Real: fragmento_rochoso Previsto: satelite_inativo Confiança: 63.10% | Risco: BAIXO
✓ Real: satelite_ativo Previsto: satelite_ativo Confiança: 99.99% | Risco: BAIXO
✓ Real: fragmento_rochoso Previsto: satelite_inativo Confiança: 63.10% | Risco: BAIXO
✓ Real: satelite_ativo Previsto: satelite_ativo Confiança: 99.99% | Risco: BAIXO
✓ Real: satelite_ativo Previsto: satelite_ativo Confiança: 99.99% | Risco: BAIXO
[GRÁFICO] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\ml\historico_treino.png

[OK] Arquivos salvos em ml/
+ ml\modelo_cnn.pth (melhor checkpoint)
+ ml\metricas.json
+ ml\historico_treino.png
+ db\orbitalguard.json (banco limpo e atualizado)
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard>
```

Terminal — python vision/analisar_orbita.py: 6/6 objetos classificados corretamente

```
Problems Output Debug Console Terminal Ports

PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python vision/analisar_orbita.py
File "D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\vision\analisar_orbita.py", line 123
    x, y, w, h = r["box"]
    ^
IndentationError: expected an indented block after 'for' statement on line 122

PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python vision/analisar_orbita.py

OrbitalGuard - Análise de Imagem Orbital
=====
[CPU] Modelo carregado (cpu)
[CENA] Grid 3x2 gerado: D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\vision\cena_orbital.png (272x188px)
[WAPA] Salvo em D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard\vision\wapa_orbital.png

[RESULTADOS] 6 objetos classificados (6/6 corretos):
Acerto Classe Real Previsto Conf Risco
-----
OK satelite_ativo satelite_ativo 98.5% BAIXO
OK detrito_metalico detrito_metalico 99.6% ALTO
Acerto Classe Real Previsto Conf Risco
-----
OK satelite_ativo satelite_ativo 98.5% BAIXO
Acerto Classe Real Previsto Conf Risco
-----
OK satelite_ativo satelite_ativo 98.5% BAIXO
OK detrito_metalico detrito_metalico 99.6% ALTO
OK fragmento_rochoso fragmento_rochoso 98.9% ALTO
OK satelite_inativo satelite_inativo 99.6% MEDIO
OK detrito_metalico detrito_metalico 98.5% ALTO
OK fragmento_rochoso fragmento_rochoso 99.1% ALTO

[COBERTURA] {'satelite_ativo': 16.7, 'detrito_metalico': 33.3, 'fragmento_rochoso': 33.3, 'satelite_inativo': 16.7}

[OK] - vision\wapa_orbital.png | vision\resultado_visao.json
PS D:\Atividades-FIAP-FIAP_IA_Online\Fase 7\Atividade Substitutiva Global Solution\OrbitalGuard>
```


Terminal — python aws/lambda_function.py: 4 cenários com CNN integrada e ISS real

```
PS D:\Atividades-FIAP-FIAP_IA\OnlineVase 7\Atividade Substitutiva Global Solution\OrbitalGuard> python aws/lambda_function.py
OrbitalGuard - AWS Lambda (Simulação Local)

[Lambda] Cenário: ISS (altitude padrão)
Status HTTP      : 200
Altitude         : 408 km | Inclinação: 51.6°
Nível de Alerta  : ALTO
Prob. Max Colisão : 69.6%
ISS Posição      : lat=-27.59 lon=134.36
Fonte ISS        : Open Notify API (real)
Detrito mais próx. : Bloco Superior Foguete (detrito_metalico)
CNN - Análise    : Bloco Superior Foguete
CNN - Tipo esperado: detrito_metalico
CNN - Classe prev. : detrito_metalico (99.9% confiança)

[Lambda] Cenário: Órbita LEO Alta
Status HTTP      : 200
Altitude         : 780 km | Inclinação: 86.4°
Nível de Alerta  : CRITICO
Prob. Max Colisão : 100.0%
ISS Posição      : lat=-27.47 lon=134.38
Fonte ISS        : Open Notify API (real)
Detrito mais próx. : Iridium 33 (inativo) (satellite_inativo)
CNN - Análise    : Iridium 33 (inativo)
CNN - Tipo esperado: satellite_inativo
CNN - Classe prev. : satellite_inativo (54.7% confiança)

[Lambda] Cenário: Órbita Sun-Sync
Status HTTP      : 200
Altitude         : 800 km | Inclinação: 98.6°
Nível de Alerta  : CRITICO
Prob. Max Colisão : 100.0%
ISS Posição      : lat=-27.31 lon=134.55
Fonte ISS        : Open Notify API (real)
Detrito mais próx. : Fragmento Fengyun-1C (detrito_metalico)
CNN - Análise    : Fragmento Fengyun-1C
CNN - Tipo esperado: detrito_metalico
CNN - Classe prev. : detrito_metalico (99.9% confiança)

[Lambda] Cenário: Órbita Baixa Crítica
Status HTTP      : 200
Altitude         : 430 km | Inclinação: 82.9°
Nível de Alerta  : CRITICO
Prob. Max Colisão : 100.0%
ISS Posição      : lat=-27.24 lon=134.62
Fonte ISS        : Open Notify API (real)
Detrito mais próx. : Fragmento Kosmos-1408 (fragmento_rochoso)
CNN - Análise    : Fragmento Kosmos-1408
CNN - Tipo esperado: fragmento_rochoso
CNN - Classe prev. : fragmento_rochoso (94.9% confiança)

Fonte ISS        : Open Notify API (real)
Detrito mais próx. : Fragmento Kosmos-1408 (fragmento_rochoso)
```

3.4 Dashboard — Evidencias de Funcionamento

Histórico de Predições — TinyDB NoSQL: 8 predições, distribuição por classe

CNN Própria (PyTorch)

Open Notify ISS API

ESP32 + MPU6050

Região simulada

Órbita LEO — 200-2000 km

Inclinação: 0-130°

Simular Risco Orbital

Altitude (km)

Inclinação (°)

Objeto para classificar (CNN)

detrito_metalico

Alan Robin Santos — RM 567437

FIAP IA | Fase 7 — Global Solution 2026.3

satellite_ativo

satellite_ativo

100.0%

BAIXO

2025-06-16T12:54:27.003035

AWS Lambda — Simulação Serverless + CNN

ISS (altitude padrão) — Alerta: ALTO

Altitude: 408.0 km

Inclinação: 51.6°

Prob. Colisão: 69.6%

CNN classificou: com 99.9% de confiança

ISS: lat=-27.59 | lon=134.36 | Open Notify API (real)

Detrito mais próximo: Bloco Superior Foguete — 69.6%

Órbita LEO Alta — Alerta: CRITICO

Altitude: 780.0 km

Inclinação: 86.4°

Prob. Colisão: 100.0%

CNN classificou: com 54.7% de confiança

ISS: lat=-27.47 | lon=134.38 | Open Notify API (real)

Detrito mais próximo: Iridium 33 (inativo) — 100.0%

Órbita Sun-Sync — Alerta: CRITICO

Órbita Baixa Crítica — Alerta: CRITICO

AWS Lambda — Cenários ISS e LEO Alta com CNN integrada e posição real da ISS

🔗 CNN Própria (PyTorch)

🌐 Open Notify ISS API

🔗 ESP32 + MPU6050

Região simulada

Órbita LEO — 200-2000 km

Inclinação: 0-130°

Simular Risco Orbital

Altitude (km)
408

Inclinação (°)
52

Objeto para classificar (CNN)
detrito_metalico

Alan Rubin Santos — RM 567437

FIAP IA | Fase 7 — Global Solution 2026.1

📄 Órbita Sun-Sync — Alerta: CRITICO

Altitude

820.0 km

Inclinação

98.6°

Prob. Colisão

100.0%

CNN classificou: 🟢 com 99.9% de confiança

ISS: lat=-27.31 | lon=134.55 | Open Notify API (real)

Detrito mais próximo: Fragmento Fengyun-1C — 100.0%

📄 Órbita Baixa Crítica — Alerta: CRITICO

Altitude

430.0 km

Inclinação

82.9°

Prob. Colisão

100.0%

CNN classificou: 🟢 com 94.9% de confiança

ISS: lat=-27.24 | lon=134.62 | Open Notify API (real)

Detrito mais próximo: Fragmento Kosmos-1408 — 100.0%

🏗️ Arquitetura da Solução

Fontes

- Imagens Sintéticas
- Open Notify ISS API

Pipeline

- Feature Engineering
- CNN (PyTorch)

Entregáveis

- Dashboard Streamlit
- Mapa Orbital

AWS Lambda — Cenários Sun-Sync e Órbita Crítica + Arquitetura da Solução

🔗 CNN Própria (PyTorch)

🌐 Open Notify ISS API

🔗 ESP32 + MPU6050

Região simulada

Órbita LEO — 200-2000 km

Inclinação: 0-130°

Simular Risco Orbital

Altitude (km)
408

Inclinação (°)
52

Objeto para classificar (CNN)
detrito_metalico

Alan Rubin Santos — RM 567437

FIAP IA | Fase 7 — Global Solution 2026.1

ISS: lat=-27.31 | lon=134.55 | Open Notify API (real)

Detrito mais próximo: Fragmento Fengyun-1C — 100.0%

📄 Órbita Baixa Crítica — Alerta: CRITICO

Altitude

430.0 km

Inclinação

82.9°

Prob. Colisão

100.0%

CNN classificou: 🟢 com 94.9% de confiança

ISS: lat=-27.24 | lon=134.62 | Open Notify API (real)

Detrito mais próximo: Fragmento Kosmos-1408 — 100.0%

🏗️ Arquitetura da Solução

Fontes

- Imagens Sintéticas
- Open Notify ISS API
- ESP32 + MPU6050

Pipeline

- Feature Engineering
- CNN (PyTorch)
- Previsão de Risco

Entregáveis

- Dashboard Streamlit
- Mapa Orbital
- Alerta de Colisão

🔗 Imagens Sintéticas + CNN (PyTorch) + Classificação de Detritos

🌐 Open Notify API + Lambda + CNN + Risco de Colisão Orbital

🔗 ESP32 + MPU6050 + MQTT + Dashboard (enriquecimento local)

Modelos: 🟢 CNN PyTorch (modelo de reconhecimento) | Dashboard: Streamlit + Plotly | Banco: PostgreSQL

4. Conclusão

O OrbitalGuard demonstrou com sucesso como redes neurais convolucionais podem ser aplicadas a classificação de detritos espaciais, respondendo diretamente ao tema da Global Solution 2026.1: tecnologias da economia espacial gerando impacto positivo na Terra.

Os cinco módulos integrados — CNN PyTorch com data augmentation, dashboard interativo com simulador orbital ao vivo, sensor IoT ESP32+MPU6050, visao computacional por grid sintético e arquitetura AWS Lambda com API real da ISS — compõem uma solução completa e coerente.

Principais contribuições técnicas:

- CNN treinada com data augmentation real — modelo aprende forma geométrica, não cor, atingindo 96.88% de acurácia com erros tecnicamente defensáveis
- model.py compartilhado entre todos os módulos — arquitetura definida em um único lugar
- Lambda conectada a CNN — classifica o tipo do detrito mais proximo, não um tipo fixo
- TinyDB NoSQL como banco de dados — conceitualmente diferente do SQLite
- ESP32 + MPU6050 detectando vibração orbital — sensor relevante ao tema espacial
- Integração com Open Notify API (ISS real) em 3 dos 4 cenários Lambda

Como próximos passos: integração com dados TLE reais via Space-Track.org, deploy da função Lambda em produção com Endpoint público, e expansão do dataset com imagens de radar real do ESA Space Debris Office.

5. Links e Referencias

Repositório GitHub

https://github.com/Yole87/Atividades-FIAP/tree/FIAP_IA_Online/Fase%207/Atividade%20Substitutiva%20Global%20Solution

Vídeo Demonstrativo

<https://youtu.be/jC8Nrqa1koA>

Referencias

- Open Notify ISS API: <http://api.open-notify.org/>
- PyTorch Documentation: <https://pytorch.org/docs/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Adafruit MPU6050: https://github.com/adafruit/Adafruit_MPU6050
- ESA Space Debris Office: https://www.esa.int/Space_Safety/Space_Debris
- TinyDB Documentation: <https://tinydb.readthedocs.io/>
- HiveMQ MQTT Broker: <https://www.hivemq.com/>
- scikit-learn: <https://scikit-learn.org/>